Unit 4

Model Development: Simple and Multiple Regression, Model Evaluation using Visualization, Residual Plot, Distribution Plot, Polynomial Regression and Pipelines, Measures for In-sample Evaluation, Prediction and Decision Making, Feature Engineering.

https://www.geeksforgeeks.org/machine-learning-model-evaluation/

Simple and Multiple Regression

Linear regression is a model that predicts one variable's values based on another's importance. It's one of the most popular and widely-used models in machine learning, and it's also one of the first things you should learn as you explore machine learning.

Linear regression is so popular because it's so simple: all it does is try to predict values based on past data, which makes it easy to get started with and understand. The simplicity means it's also easy to implement, which makes it a great starting point if you're new to machine learning.

There are two types of linear regression algorithms -

- Simple - deals with two features.
- Multiple - deals with more than two features.

Simple Linear Regression in Machine Learning

Simple Linear Regression is a type of Regression algorithms that models the relationship between a dependent variable and a single independent variable. The relationship shown by a Simple Linear Regression model is linear or a sloped straight line, hence it is called Simple Linear Regression.

The key point in Simple Linear Regression is that the ***dependent variable must be a continuous/real value***. However, the independent variable can be measured on continuous or categorical values.

Simple Linear regression algorithm has mainly two objectives:

- **Model the relationship between the two variables.** Such as the relationship between Income and expenditure, experience and Salary, etc.
- **Forecasting new observations.** Such as Weather forecasting according to temperature, Revenue of a company according to the investments in a year, etc.

Simple Linear Regression Model:

The Simple Linear Regression model can be represented using the below equation:

$y= a_0+a_1x+ \varepsilon$

Where,

a0= It is the intercept of the Regression line (can be obtained putting x=0)
a1= It is the slope of the regression line, which tells whether the line is increasing or decreasing.
ε = The error term. (For a good model it will be negligible)

Implementation of Simple Linear Regression Algorithm using Python

Problem Statement example for Simple Linear Regression:

Here we are taking a dataset that has two variables: salary (dependent variable) and experience (Independent variable). The goals of this problem is:

- **We want to find out if there is any correlation between these two variables**
- **We will find the best fit line for the dataset.**
- **How the dependent variable is changing by changing the independent variable.**

What Is Multiple Linear Regression (MLR)?

One of the most common types of predictive analysis is multiple linear regression. This type of analysis allows you to understand the relationship between a continuous dependent variable and two or more independent variables.

The independent variables can be either continuous (like age and height) or categorical (like gender and occupation). It's important to note that if your dependent variable is categorical, you should dummy code it before running the analysis.

Formula and Calculation of Multiple Linear Regression

Several circumstances that influence the dependent variable simultaneously can be controlled through multiple regression analysis. Regression analysis is a method of analyzing the relationship between independent variables and dependent variables.

Let k represent the number of variables denoted by x1, x2, x3, ……, xk.

For this method, we assume that we have k independent variables x1, . . . , xk that we can set, then they probabilistically determine an outcome Y.

Furthermore, we assume that Y is linearly dependent on the factors according to

$Y = \beta 0 + \beta 1x1 + \beta 2x2 + \cdot \cdot \cdot + \beta kxk + \varepsilon$

- The variable yi is dependent or predicted
- The slope of y depends on the y-intercept, that is, when xi and x2 are both zero, y will be $\beta 0$.
- The regression coefficients $\beta 1$ and $\beta 2$ represent the change in y as a result of one-unit changes in xi1 and xi2.
- βp refers to the slope coefficient of all independent variables
- ε term describes the random error (residual) in the model.

Where ε is a standard error, this is just like we had for simple linear regression, except k doesn't have to be 1.

We have n observations, n typically being much more than k.

For i th observation, we set the independent variables to the values xi1, xi2 . . . , xik and measure a value yi for the random variable Yi.

Thus, the model can be described by the equations.

$Yi = \beta 0 + \beta 1 xi1 + \beta 2 xi2 + \cdots + \beta k xik + i$ for i = 1, 2, . . . , n,

Where the errors i are independent standard variables, each with mean 0 and the same unknown variance σ2.

Altogether the model for multiple linear regression has k + 2 unknown parameters:

β0, β1, . . . , βk, and σ 2.

When k was equal to 1, we found the least squares line $y = \hat{\beta} 0 + \hat{\beta} 1x$.

It was a line in the plane R 2.

Now, with k ≥ 1, we'll have a least squares hyperplane.

$y = \hat{\beta} 0 + \hat{\beta} 1x1 + \hat{\beta} 2x2 + \cdots + \hat{\beta} kxk$ in Rk+1.

The way to find the estimators $\hat{\beta} 0$, $\hat{\beta} 1$, . . ., and $\hat{\beta} k$ is the same.

Take the partial derivatives of the squared error.

$Q = Xn\ i=1\ (yi - (\beta 0 + \beta 1 xi1 + \beta 2 xi2 + \cdots + \beta kxik))2$

When that system is solved we have fitted values

$\hat{y}i = \hat{\beta} 0 + \hat{\beta} 1xi1 + \hat{\beta} 2xi2 + \cdots + \hat{\beta} kxik$ for i = 1, . . . , n that should be close to the actual values yi.

Assumptions of Multiple Linear Regression

In multiple linear regression, the dependent variable is the outcome or result from you're trying to predict. The independent variables are the things that explain your dependent variable. You can use them to build a model that accurately predicts your dependent variable from the independent variables.

For your model to be reliable and valid, there are some essential requirements:

- The independent and dependent variables are linearly related.
- There is no strong correlation between the independent variables.
- Residuals have a constant variance.
- Observations should be independent of one another.
- It is important that all variables follow multivariate normality.

Example of How to Use Multiple Linear Regression

```
from sklearn.datasets import load_boston

import pandas as pd

from sklearn.model_selection import train_test_split

def sklearn_to_df(data_loader):

    X_data = data_loader.data

    X_columns = data_loader.feature_names

    X = pd.DataFrame(X_data, columns=X_columns)

    y_data = data_loader.target

    y = pd.Series(y_data, name='target')

    return x, y

x, y = sklearn_to_df(load_boston())

x_train, x_test, y_train, y_test = train_test_split(

    x, y, test_size=0.2, random_state=42)

from load_dataset import x_train, x_test, y_train, y_test
```

```
from multiple_linear_regression import MultipleLinearRegression

from sklearn.linear_model import LinearRegression

mulreg = MultipleLinearRegression()

# fit our LR to our data

mulreg.fit(x_train, y_train)

# make predictions and score

pred = mulreg.predict(x_test)

# calculate r2_score

score = mulreg.r2_score(y_test, pred)

print(f'Our Final R^2 score: {score}')
```

The Difference Between Linear and Multiple Regression

When predicting a complex process's outcome, it is best to use multiple linear regression instead of simple linear regression.

A simple linear regression can accurately capture the relationship between two variables in simple relationships. On the other hand, multiple linear regression can capture more complex interactions that require more thought.

A multiple regression model uses more than one independent variable. It does not suffer from the same limitations as the simple regression equation, and it is thus able to fit curved and non-linear relationships. The following are the uses of multiple linear regression.

1. Planning and Control.
2. Prediction or Forecasting.

Estimating relationships between variables can be exciting and useful. As with all other regression models, the multiple regression model assesses relationships among variables in terms of their ability to predict the value of the dependent variable.

Data visualization is actually a set of data points and information that are represented graphically to make it easy and quick for user to understand. Data visualization is good if it has a clear meaning, purpose, and is very easy to interpret, without requiring context. Tools of data visualization provide an accessible way to see and understand trends, outliers, and patterns in data by using visual effects or elements such as a chart, graphs, and maps.

Characteristics of Effective Graphical Visual :

- It shows or visualizes data very clearly in an understandable manner.
- It encourages viewers to compare different pieces of data.
- It closely integrates statistical and verbal descriptions of data set.
- It grabs our interest, focuses our mind, and keeps our eyes on message as human brain tends to focus on visual data more than written data.
- It also helps in identifying area that needs more attention and improvement.
- Using graphical representation, a story can be told more efficiently. Also, it requires less time to understand picture than it takes to understand textual data.

Categories of Data Visualization;

Data visualization is very critical to market research where both numerical and categorical data can be visualized that helps in an increase in impacts of insights and also helps in reducing risk of analysis paralysis. So, data visualization is categorized into following categories:

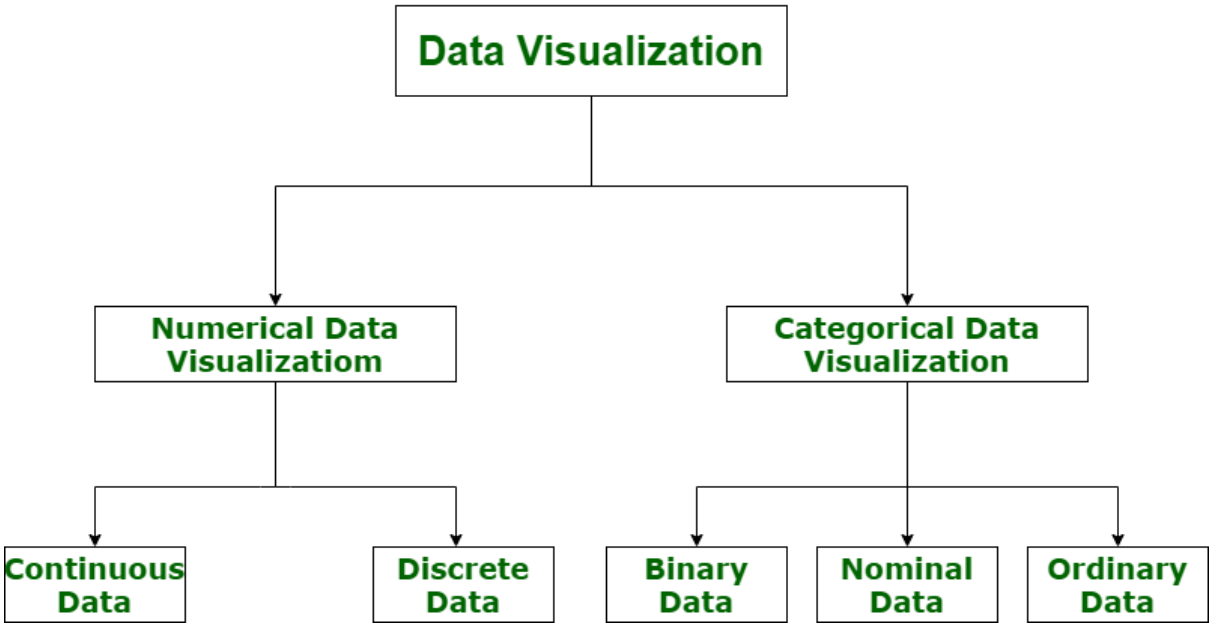


Figure – Categories of Data Visualization

Machine Learning Model does not require hard-coded algorithms. We feed a large amount of data to the model and the model tries to figure out the features on its own to make future predictions. So we must also use some techniques to determine the predictive power of the model.

Machine Learning Model Evaluation

Model evaluation is the process that uses some metrics which help us to analyze the performance of the model. As we all know that model development is a multi-step process and a check should be kept on how well the model generalizes future predictions. Therefore evaluating a model plays a vital role so that we can judge the performance of our model. The evaluation also helps to analyze a model's key weaknesses. There are many metrics like Accuracy, Precision, Recall, F1 score, Area under Curve, Confusion Matrix, and Mean Square Error. Cross Validation is one technique that is followed during the training phase and it is a model evaluation technique as well.

Cross Validation and Holdout

Cross Validation is a method in which we do not use the whole dataset for training. In this technique, some part of the dataset is reserved for testing the model. There are many types of Cross-Validation out of which K Fold Cross Validation is mostly used. In K Fold Cross Validation the original dataset is divided into k subsets. The subsets are known as folds. This is repeated k times where 1 fold is used for testing purposes. Rest k-1 folds are used for training the model. So each data point acts as a test subject for the model as well as acts as the training subject. It is seen that this technique generalizes the model well and reduces the error rate

Holdout is the simplest approach. It is used in neural networks as well as in many classifiers. In this technique, the dataset is divided into train and test datasets. The dataset is usually divided into ratios like 70:30 or 80:20. Normally a large percentage of data is used for training the model and a small portion of the dataset is used for testing the model.

Evaluation Metrics for Classification Task

In this Python code, we have imported the iris dataset which has features like the length and width of sepals and petals. The target values are Iris setosa, Iris virginica, and Iris versicolor. After importing the dataset we divided the dataset into train and test datasets in the ratio 80:20. Then we called Decision Trees and trained our model. After that, we performed the prediction and calculated the accuracy score, precision, recall, and f1 score. We also plotted the confusion matrix.

Importing Libraries and Dataset

Python libraries make it very easy for us to handle the data and perform typical and complex tasks with a single line of code.

- **Pandas** – This library helps to load the data frame in a 2D array format and has multiple functions to perform analysis tasks in one go.
- **Numpy** – Numpy arrays are very fast and can perform large computations in a very short time.
- **Matplotlib**/**Seaborn** – This library is used to draw visualizations.
- Sklearn – This module contains multiple libraries having pre-implemented functions to perform tasks from data preprocessing to model development and evaluation.

Accuracy

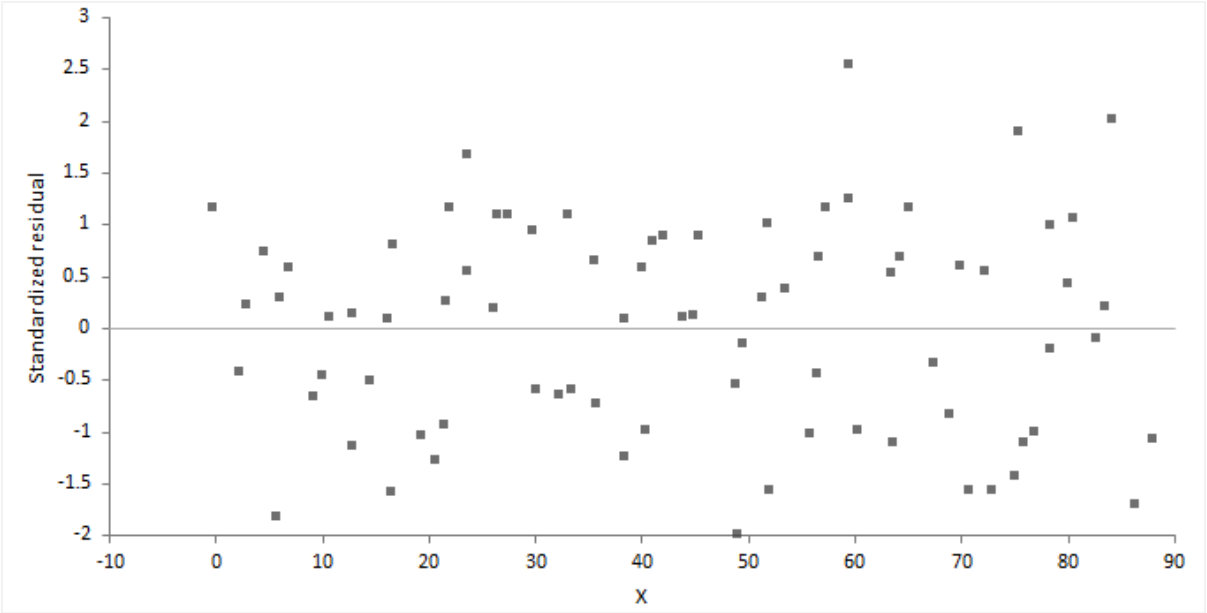
Accuracy is defined as the ratio of the number of correct predictions to the total number of predictions. This is the most fundamental metric used to evaluate the model. The formula is given by

Accuracy = (TP+TN)/(TP+TN+FP+FN)

However, Accuracy has a drawback. It cannot perform well on an imbalanced dataset. Suppose a model classifies that the majority of the data belongs to the major class label. It yields higher accuracy. But in general, the model cannot classify on minor class labels and has poor performance.

Residual plot

A residual plot shows the difference between the observed response and the fitted response values. The ideal residual plot, called the null residual plot, shows a random scatter of points forming an approximately constant width band around the identity line.



It is important to check the fit of the model and assumptions – constant variance, normality, and independence of the errors, using the residual plot, along with normal, sequence, and lag plot.

Assumption	How to check
Model function is linear	The points form a pattern when the model function is incorrect. You might be able to transform variables or add polynomial and interaction terms to remove the pattern.
Constant variance	If the points tend to form an increasing, decreasing or non-constant width band, then the variance is not constant. You should consider transforming the response variable or incorporating weights into the model. When variance increases as a percentage of the response, you can use a log transform, although you should ensure it does not produce a poorly fitting model. Even with non-constant variance, the parameter estimates remain unbiased if somewhat inefficient. However, the hypothesis tests and confidence intervals are inaccurate.
Normality	Examine the normal plot of the residuals to identify non-normality. Violation of the normality assumption only becomes an issue with small sample sizes. For large sample sizes, the assumption is less important due

	to the central limit theorem, and the fact that the F- and t-tests used for hypothesis tests and forming confidence intervals are quite robust to modest departures from normality.
Independence	When the order of the cases in the dataset is the order in which they occurred: Examine a sequence plot of the residuals against the order to identify any dependency between the residual and time. Examine a lag-1 plot of each residual against the previous residual to identify a serial correlation, where observations are not independent, and there is a correlation between an observation and the previous observation. Time-series analysis may be more suitable to model data where serial correlation is present.

For a model with many terms, it can be difficult to identify specific problems using the residual plot. A non-null residual plot indicates that there are problems with the model, but not necessarily what these are.

Distribution Plots

Seaborn is a Python data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. This article deals with the distribution plots in seaborn which is used for examining univariate and bivariate distributions. In this article we will be discussing 4 types of distribution plots namely:

1. joinplot
2. distplot
3. pairplot
4. rugplot

Besides providing different kinds of visualization plots, seaborn also contains some built-in datasets. We will be using the tips dataset in this article. The “tips” dataset contains information about people who probably had food at a restaurant and whether or not they left a tip, their age, gender and so on. Lets have a look at it. **Code :**

```
# import the necessary libraries
import seaborn as sns
import matplotlib.pyplot as plt % matplotlib inline

# to ignore the warnings
from warnings import filterwarnings

# load the dataset
df = sns.load_dataset('tips')

# the first five entries of the dataset
df.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Now, lets proceed onto the plots.

Displot
It is used basically for univariant set of observations and visualizes it through a histogram i.e. only one observation and hence we choose one particular column of the dataset. **Syntax:**
Now, lets proceed onto the plots.

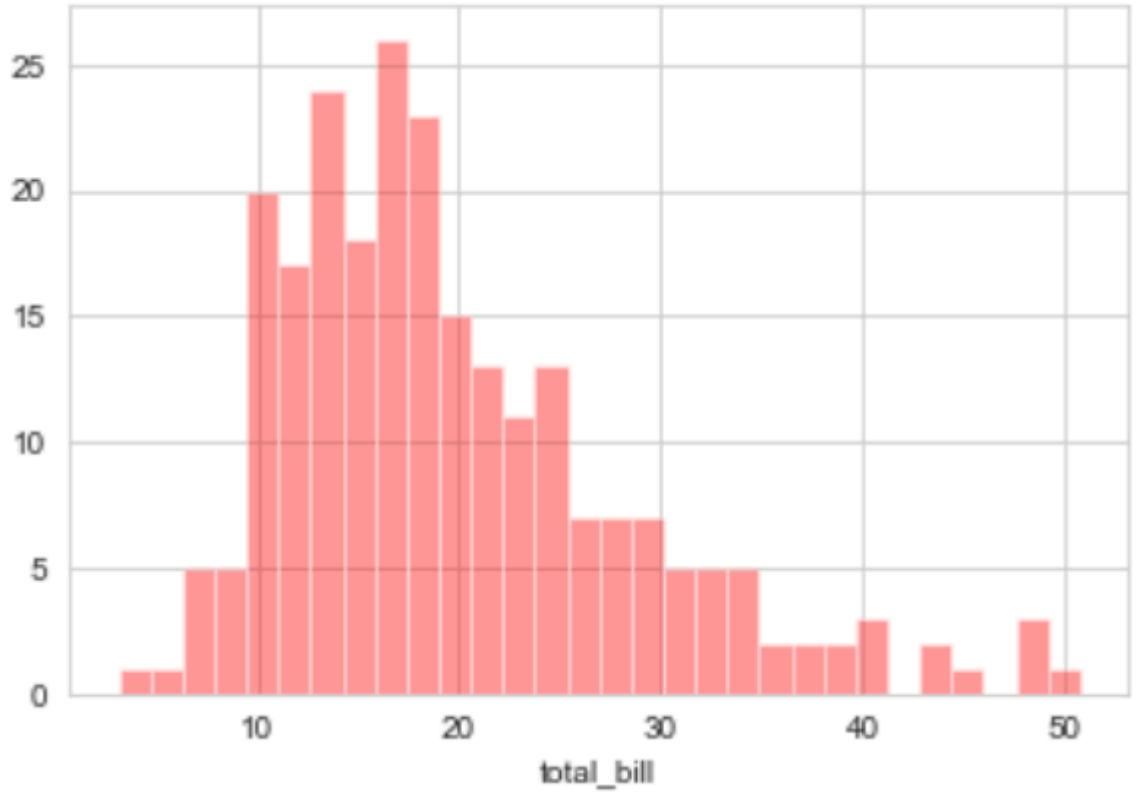
Displot
It is used basically for univariant set of observations and visualizes it through a histogram i.e. only one observation and hence we choose one particular column of the dataset. **Syntax:**
distplot(a[, bins, hist, kde, rug, fit, ...])

Example:

- Python3

```
# set the background style of the plot
sns.set_style('whitegrid')
sns.distplot(df['total_bill'], kde = False, color ='red', bins = 30)
```

Output:



Explanation:

- KDE stands for **Kernel Density Estimation** and that is another kind of the plot in seaborn.
- bins is used to set the number of bins you want in your plot and it actually depends on your dataset.
- color is used to specify the color of the plot

Now looking at this we can say that most of the total bill given lies between 10 and 20.

Joinplot

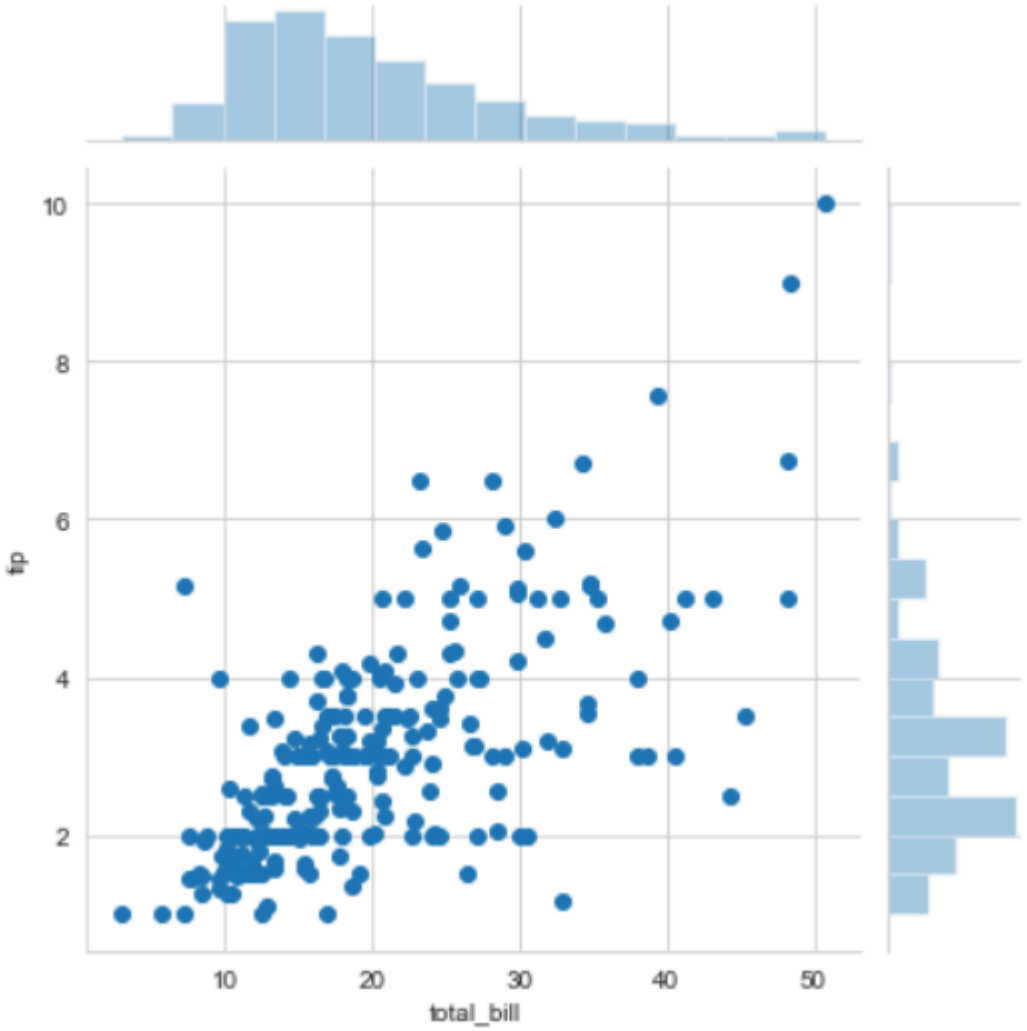
It is used to draw a plot of two variables with bivariate and univariate graphs. It basically combines two different plots. **Syntax:**

jointplot(x, y[, data, kind, stat_func, ...])

Example:

- Python3

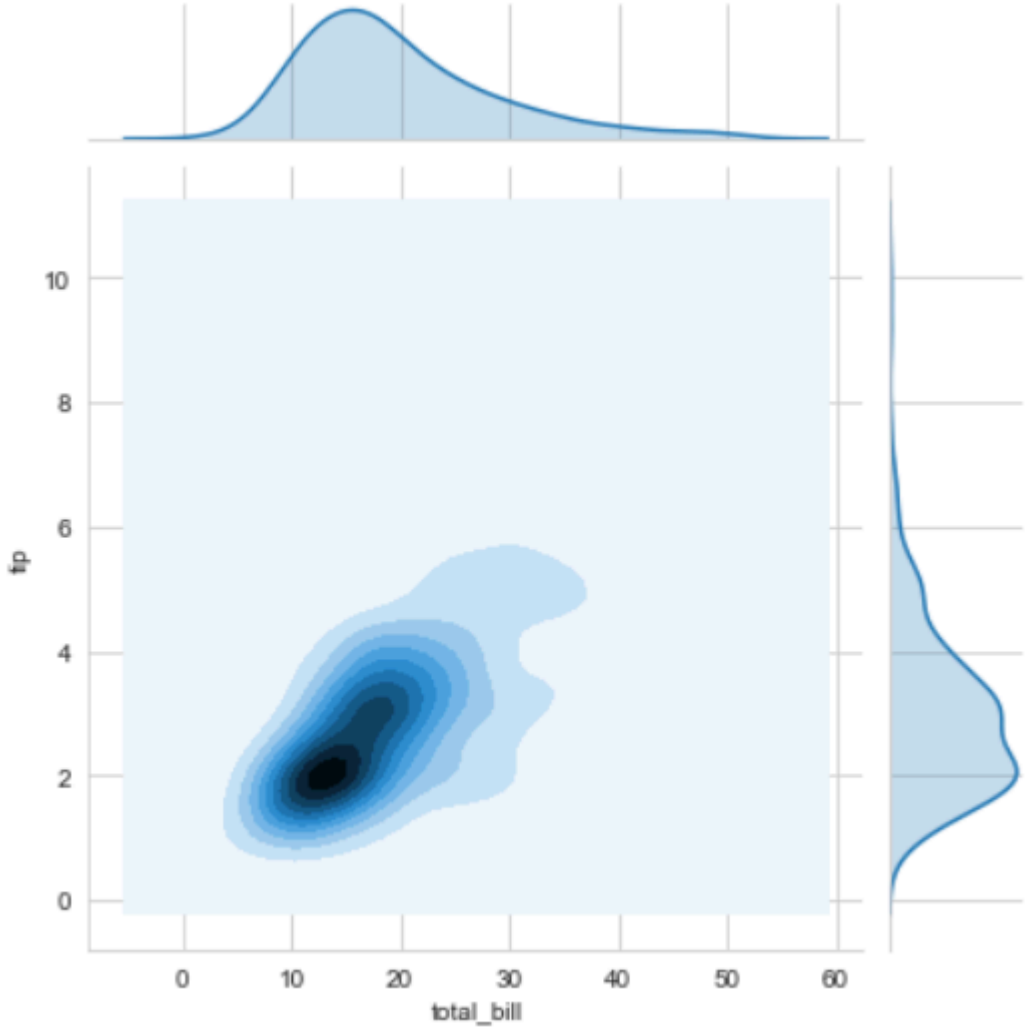
```
sns.jointplot(x ='total_bill', y ='tip', data = df)
```



Output:

- Python3

```
sns.jointplot(x ='total_bill', y ='tip', data = df, kind ='kde')
# KDE shows the density where the points match up the most
```



Explanation:

- kind is a variable that helps us play around with the fact as to how do you want to visualise the data.It helps to see whats going inside the joinplot. The default is scatter and can be hex, reg(regression) or kde.
- x and y are two strings that are the column names and the data that column contains is used by specifying the data parameter.
- here we can see tips on the y axis and total bill on the x axis as well as a linear relationship between the two that suggests that the total bill increases with the tips.
    - hue sets up the categorical separation between the entries if the dataset.
    - palette is used for designing the plots.

Polynomial Regression with a Machine Learning Pipeline

Welcome back! It's very exciting to apply the knowledge that we already have to build machine learning models with some real data. ***Polynomial Regression***, the topic that we discuss today, is such a model which may require some complicated workflow depending on the problem statement and the dataset.

Today, we discuss how to build a Polynomial Regression Model, and how to preprocess the data before making the model. Actually, we apply a series of steps in a particular order to build the

complete model. All the necessary tools are available in Python Scikit-learn Machine Learning library.

Prerequisites

If you're not familiar with Python, numpy, pandas, machine learning and Scikit-learn, please read my previous articles that are prerequisites for this article.

- Principal Component Analysis (PCA) with Scikit-learn
- Linear Regression with Gradient Descent
- NumPy for Data Science: Part 1
- pandas for Data Science: Part 1

Without further delay, let's go into the problem definition.

Problem Definition

We have a dataset which contains information about **Age**, **Height**, and **Weight** of 71 people. We want to build a regression model which captures the Weight of a person depending on Age and Height and evaluate its performance. Then we use that model to make predictions for new cases. Actually, we don't know the nature or complexity of our data until we plot them. Sometimes, we can easily fit a straight line and describe the model. But most of the time, this is not the case for real-world data. The data may be complicated and you need to consider a different approach to tackle the problem.

Actually, we don't know the nature or complexity of our data until we plot them. Sometimes, we can easily fit a straight line and describe the model. But most of the time, this is not the case for real-world data. The data may be complicated and you need to consider a different approach to tackle the problem.

The Dataset

We have a dataset called ***person_data.csv*** (download here) which contains information about Age, Height, and Weight of 71 people. Let's load it using the pandas ***read_csv()*** function and store it in the ***df*** variable.

The Dataset

We have a dataset called ***person_data.csv*** (download here) which contains information about Age, Height, and Weight of 71 people. Let's load it using the pandas ***read_csv()*** function and store it in the ***df*** variable.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline
```

```
df = pd.read_csv('person_data.csv')
df.head()
```

	Age	Height	Weight
0	10	138	23.0
1	11	138	22.0
2	12	138	23.5
3	13	139	24.0
4	14	139	26.0

```
df.shape
```

```
(71, 3)
```

Let's check whether the data has missing values.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Age     71 non-null     int64  
 1   Height  71 non-null     int64  
 2   Weight  71 non-null     float64
dtypes: float64(1), int64(2)
memory usage: 1.8 KB
```

Great! All the values of the 3 columns have 71 non-null values meaning that there are no missing values.

Let's define our feature matrix and the target vector. According to the problem definition, the feature matrix — **X** contains the values of Age and Height. The target vector — **y** contains the values Weight.

Building a Regression Model is a supervised learning task so that we map the input **X** to the output **y=f(X)**.

```
X = df[['Age', 'Height']]
print(X.shape) #2-dimensional
print(type(X))
```

```
(71, 2)
<class 'pandas.core.frame.DataFrame'>
```

```
y = df['Weight']
print(y.shape) #1-dimensional
print(type(y))
```

```
(71,)
<class 'pandas.core.series.Series'>
```

The dimension of our data is 2 because **X** is 2-dimensional. So, how can we plot 2-dimensional data of **X** with **y**? Obviously, we need to create a 3D plot. But there is another way. We can combine Age and Height into one variable called **Z** and then plot **Z** with **y** in a 2D plot. Combining Age and Height into one variable by reducing the number of features in the **X** is called the ***Dimensionality Reduction*** and the technique that we use to perform dimensionality reduction is the ***Principal Component Analysis (PCA)*** which uses the correlation of these two variables.

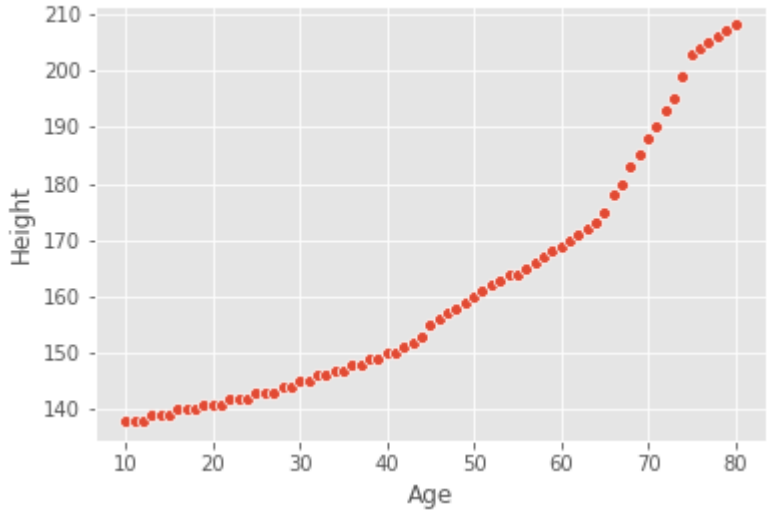
We have two advantages of performing Dimensionality Reduction before making the model:

1. It is extremely useful for visualizing our data in a 2D plot which makes easier to see important patterns

2. It is extremely useful to remove correlated variables in the feature matrix, **X**.— doing so will avoid misleading predictions

Let’s check whether Age and Height are correlated or not.

```
plt.style.use('ggplot')
sns.scatterplot(x='Age', y='Height', data=df)
plt.savefig('scatterplot1.png')
```



It seems that the two features are highly correlated. The condition when there is a significant dependency or association between the independent variables (the variables in the feature matrix **X**) is called ***multicollinearity***. Having multicollinearity will lead to misleading predictions.

Before making the model, our next task is to remove these correlated variables. As I said earlier, we use Principal Component Analysis (PCA) to do this. But before running PCA, it is essential to perform feature scaling if there is a significant difference in the scale between the features of the dataset. This is because PCA is very sensitive to the relative ranges of the original features. So, We apply ***z-score standardization*** to get all features into the same scale by using Scikit-learn **StandardScaler()** class which is in the *preprocessing* submodule in Scikit-learn.

Note: We apply feature scaling only for the feature matrix **X**. We don't need to apply it for our target vector **y**.

So, the workflow for building our model is as follows. It contains a series of steps that should be applied in the given order. Building a machine learning model is not a one time task and you may come back to an earlier step and make some modifications and then you go again through the next steps.

The general workflow is:

1. Apply feature scaling for the feature matrix X [Use Scikit-learn **StandardScaler()** class].

2. Run PCA algorithm [Use Scikit-learn **PCA()** class].

3. Create a scatterplot and identify the nature of the relationship [Use Seaborn **scatterplot()** function].

4. Can we fit a straight line to our data or do we need to add polynomial features to fit the model accurately?

5. If we can fit a straight line to our data, then run Linear Regression algorithm [Use Scikit-learn **LinearRegression()** class]

6. If we cannot fit a straight line to our data, then we need to add polynomial features [Use Scikit-learn **PolynomialFeatures()** class]

7. After adding the polynomial features, run Linear Regression algorithm [Use Scikit-learn **LinearRegression()** class]

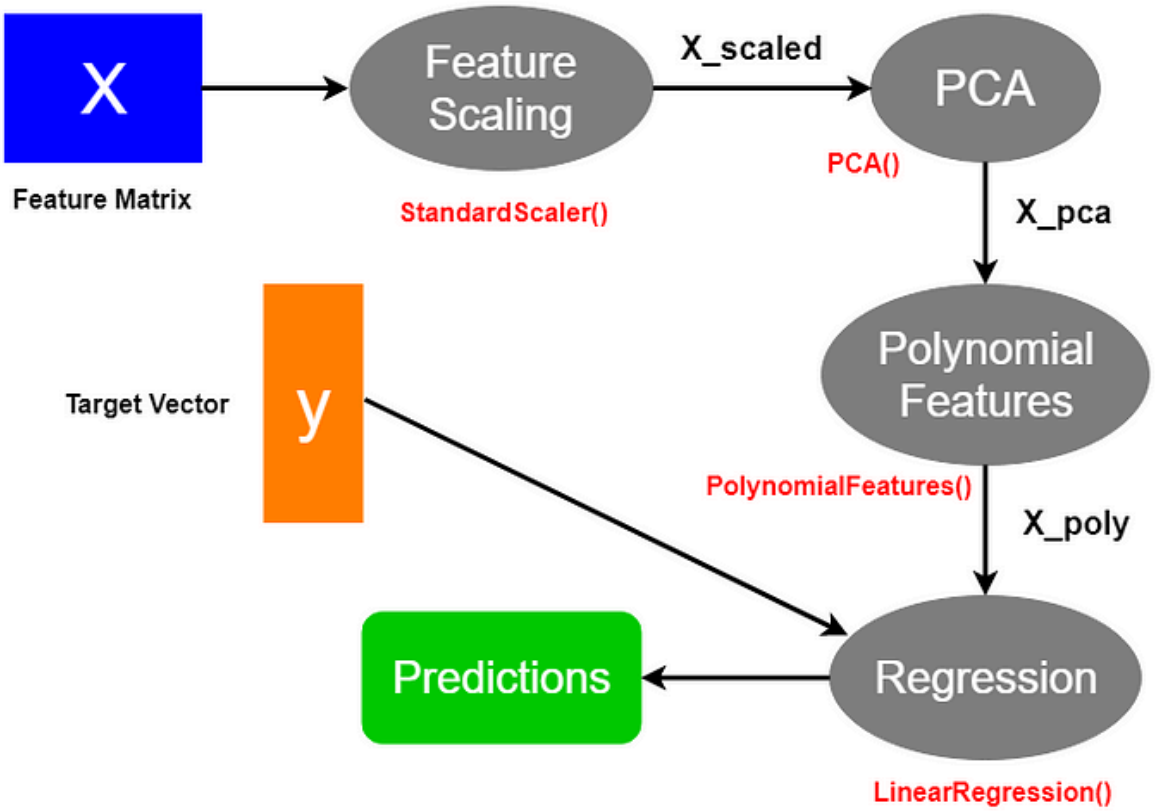


Image by Author

Apply feature scaling

We apply ***z-score standardization*** to get all features into the same scale by using Scikit-learn **StandardScaler()**. All the features are scaled according to the following formula.

$$Z = \frac{X - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc
```

```
StandardScaler(copy=True, with_mean=True, with_std=True)
```

```
X_scaled = sc.fit_transform(X)
print(X_scaled.shape)
print(type(X_scaled))
```

```
(71, 2)
<class 'numpy.ndarray'>
```

The scaled values of **X** is stored in the ***X_scaled*** variable which is a 2-dimensional numpy array.

Let's check the mean and standard deviation of the scaled **X** values.

```
print('Mean of each scaled variable')
np.mean(X_scaled, axis=0)
```

```
Mean of each scaled variable

array([3.12738880e-18, 2.50191104e-16])
```

```
print('Std of each scaled variable')
np.std(X_scaled, axis=0)
```

```
Std of each scaled variable

array([1., 1.])
```

Now, you can see that mean is zero and the standard deviation is 1 for each variable in the ***X_scaled*** matrix.

Model Evaluation Metrics in Machine Learning

Predictive models have become a trusted advisor to many businesses and for a good reason. These models can "foresee the future", and there are many different methods available, meaning any industry can find one that fits their particular challenges.

When we talk about predictive models, we are talking either about a **regression model** (continuous output) or a **classification model** (nominal or binary output). In classification problems, we use two types of algorithms (dependent on the kind of output it creates):

1. **Class output**: Algorithms like SVM and KNN create a class output. For instance, in a binary classification problem, the outputs will be either 0 or 1. However, today we have algorithms that can convert these class outputs to probability.
2. **Probability output**: Algorithms like Logistic Regression, Random Forest, Gradient Boosting, Adaboost, etc. give probability outputs. Converting probability outputs to class output is just a matter of creating a threshold probability.

Introduction

While data preparation and training a machine learning model is a key step in the machine learning pipeline, it's equally important to measure the performance of this trained model. How well the model generalizes on the unseen data is what defines adaptive vs non-adaptive machine learning models. By using different metrics for performance evaluation, we should be in a position to improve the overall predictive power of our model before we roll it out for production on unseen data.

Without doing a proper evaluation of the ML model using different metrics, and depending only on accuracy, it can lead to a problem when the respective model is deployed on unseen data and can result in poor predictions.

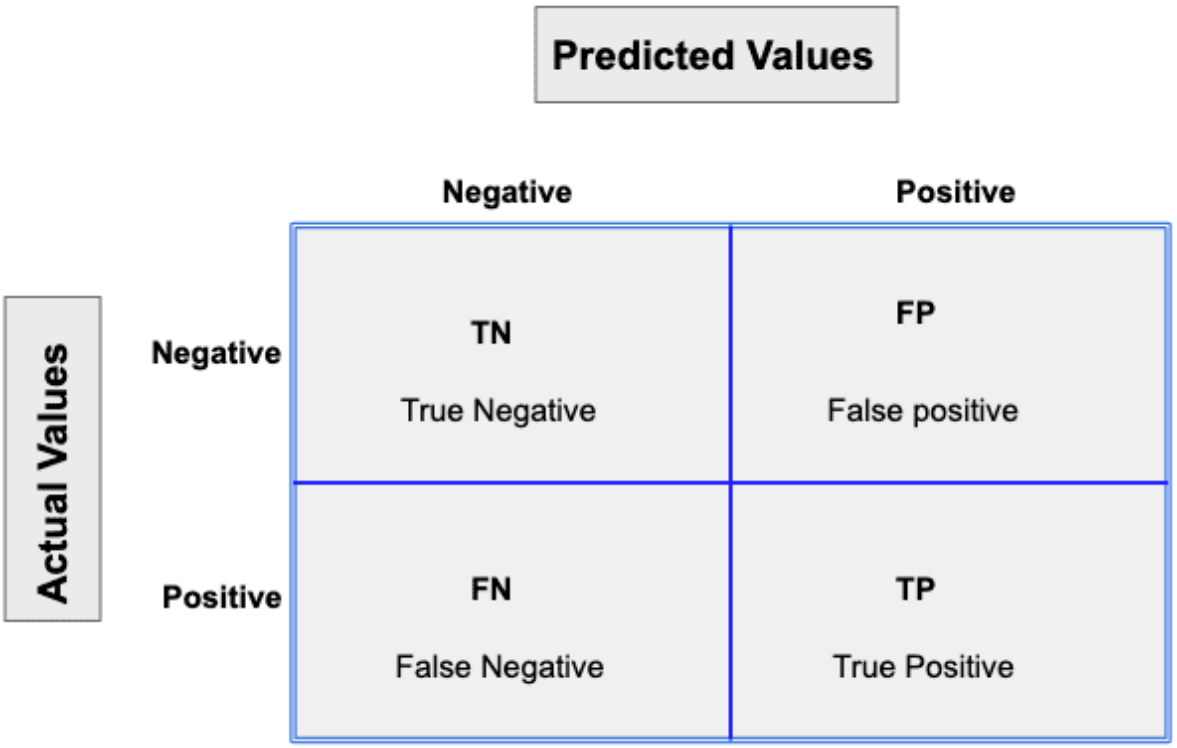
This happens because, in cases like these, our models don't learn but instead memorize; hence, they cannot generalize well on unseen data.

Model Evaluation Metrics

Let us now define the evaluation metrics for evaluating the performance of a machine learning model, which is an integral component of any data science project. It aims to estimate the generalization accuracy of a model on the future (unseen/out-of-sample) data.

Confusion Matrix

A confusion matrix is a matrix representation of the prediction results of any binary testing that is often used to **describe the performance of the classification model** (or "classifier") on a set of test data for which the true values are known.
The confusion matrix itself is relatively simple to understand, but the related terminology can be confusing.



Confusion matrix with 2 class labels.

Each prediction can be one of the four outcomes, based on how it matches up to the actual value:

- **True Positive (TP):** Predicted True and True in reality.
- **True Negative (TN):** Predicted False and False in reality.
- **False Positive (FP):** Predicted True and False in reality.
- **False Negative (FN):** Predicted False and True in reality.

Now let us understand this concept using hypothesis testing.

A **Hypothesis** is speculation or theory based on insufficient evidence that lends itself to further testing and experimentation. With further testing, a hypothesis can usually be proven true or false.
A **Null Hypothesis** is a hypothesis that says there is no statistical significance between the two variables in the hypothesis. It is the hypothesis that the researcher is trying to disprove.
We would always reject the null hypothesis when it is false, and we would accept the null hypothesis when it is indeed true.

Even though hypothesis tests are meant to be reliable, **there are two types of errors that can occur.** These errors are known as **Type 1 and Type II errors.**
For example, when examining the effectiveness of a drug, the null hypothesis would be that the drug does not affect a disease.

Type I Error:- equivalent to False Positives(FP).
The first kind of error that is possible involves the rejection of a null hypothesis that is true.

Let's go back to the example of a drug being used to treat a disease. If we reject the null hypothesis in this situation, then we claim that the drug does have some effect on a disease. But if the null hypothesis is true, then, in reality, the drug does not combat the disease at all. The drug is falsely claimed to have a positive effect on a disease.

Type II Error:- equivalent to False Negatives(FN).
The other kind of error that occurs when we accept a false null hypothesis. This sort of error is called a type II error and is also referred to as an error of the second kind.

If we think back again to the scenario in which we are testing a drug, what would a type II error look like? A type II error would occur if we accepted that the drug hs no effect on disease, but in reality, it did.

A sample python implementation of the Confusion matrix.

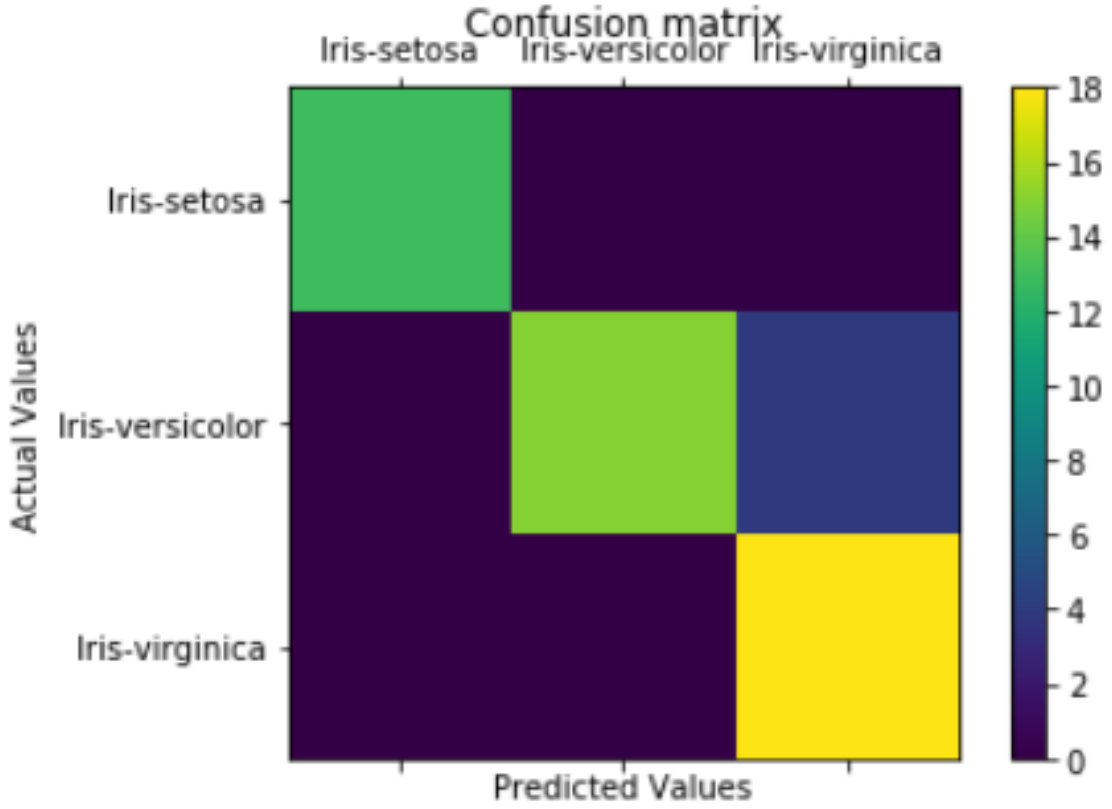
```
import warnings

import pandas as pd

from sklearn import model_selection

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import confusion_matrix

import matplotlib.pyplot as plt

%matplotlib inline
```

```
#ignore warnings

warnings.filterwarnings('ignore')

# Load digits dataset

url = "http://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"

df = pd.read_csv(url)

# df = df.values

X = df.iloc[:,0:4]

y = df.iloc[:,4]

#test size

test_size = 0.33

#generate the same set of random numbers

seed = 7

#Split data into train and test set.

X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=test_size, random_st
ate=seed)

#Train Model

model = LogisticRegression()

model.fit(X_train, y_train)

pred = model.predict(X_test)


#Construct the Confusion Matrix

labels = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']

cm = confusion_matrix(y_test, pred, labels)

print(cm)

fig = plt.figure()

ax = fig.add_subplot(111)

cax = ax.matshow(cm)
```

```
plt.title('Confusion matrix')

fig.colorbar(cax)

ax.set_xticklabels([''] + labels)

ax.set_yticklabels([''] + labels)

plt.xlabel('Predicted Values')

plt.ylabel('Actual Values')

plt.show()
```

view rawconfusion+matrix.py hosted with ❤ by GitHub

```
[[13  0  0]
 [ 0 15  4]
 [ 0  0 18]]
```



Confusion matrix with 3 class labels.

The diagonal elements represent the number of points for which the predicted label is equal to the true label, while anything off the diagonal was mislabeled by the classifier. Therefore, the higher the diagonal values of the confusion matrix the better, indicating many correct predictions.

In our case, the classifier predicted all the 13 setosa and 18 virginica plants in the test data perfectly. However, it incorrectly classified 4 of the versicolor plants as virginica.

There is also a list of rates that are often computed from a confusion matrix for a binary classifier:

1. Accuracy

Overall, how often is the classifier correct?
Accuracy = (TP+TN)/total

When our classes are roughly equal in size, we can use accuracy**,** which will give us correctly classified values.
Accuracy is a common evaluation metric for classification problems. It's the number of correct predictions made as a ratio of all predictions made.

Misclassification Rate(Error Rate): Overall, how often is it wrong. Since accuracy is the percent we correctly classified (success rate), it follows that our error rate (the percentage we got wrong) can be calculated as follows:
Misclassification Rate = (FP+FN)/total

We use the sklearn module to compute the accuracy of a classification task, as shown below.

```
#import modules

import warnings

import pandas as pd

import numpy as np

from sklearn import model_selection

from sklearn.linear_model import LogisticRegression

from sklearn import datasets

from sklearn.metrics import accuracy_score

#ignore warnings

warnings.filterwarnings('ignore')

# Load digits dataset

iris = datasets.load_iris()

# # Create feature matrix

X = iris.data

# Create target vector

y = iris.target

#test size
```

```
test_size = 0.33

#generate the same set of random numbers

seed = 7

#cross-validation settings

kfold = model_selection.KFold(n_splits=10, random_state=seed)

#Model instance

model = LogisticRegression()

#Evaluate model performance

scoring = 'accuracy'

results = model_selection.cross_val_score(model, X, y, cv=kfold, scoring=scoring)

print('Accuracy -val set: %.2f%% (%.2f)' % (results.mean()*100, results.std()))


#split data

X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=test_size, random_st
ate=seed)

#fit model

model.fit(X_train, y_train)

#accuracy on test set

result = model.score(X_test, y_test)

print("Accuracy - test set: %.2f%%" % (result*100.0))
```

view rawclassification_Accuracy.py hosted with ❤ by GitHub

The classification accuracy is **88%** on the validation set.

2. Precision

When it predicts yes, how often is it correct?
Precision=TP/predicted yes

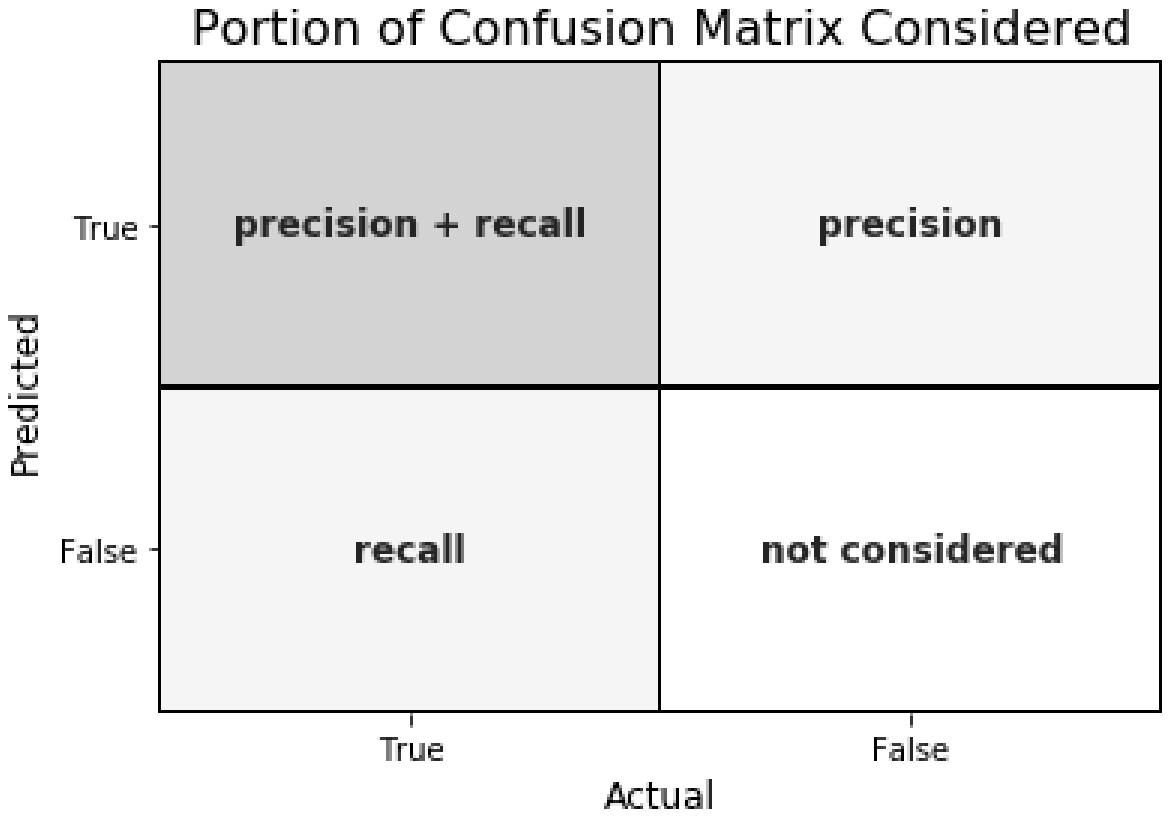
When we have a class imbalance, accuracy can become an unreliable metric for measuring our performance. For instance, if we had a 99/1 split between two classes, A and B, where the rare event, B, is our positive class, we could build a model that was 99% accurate by just saying everything belonged to class A. Clearly, we shouldn't bother building a model if it doesn't do anything to identify class B; thus, we need different metrics that will discourage this behavior. For this, we use precision and recall instead of accuracy.

3. Recall or Sensitivity

When it's actually yes, how often does it predict yes?
True Positive Rate = TP/actual yes

Recall gives us the **true positive rate** (**TPR**), which is the ratio of true positives to everything positive.
In the case of the 99/1 split between classes A and B, the model that classifies everything as A would have a recall of 0% for the positive class, B (precision would be undefined — 0/0). Precision and recall provide a better way of evaluating model performance in the face of a class imbalance. They will correctly tell us that the model has little value for our use case.

Just like accuracy, both precision and recall are easy to compute and understand but require thresholds. Besides, precision and recall only consider half of the confusion matrix:



4. F1 Score

The F1 score is the harmonic mean of the precision and recall, where an F1 score reaches its best value at 1 (perfect precision and recall) and worst at 0.

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall}$$

Why harmonic mean? Since the harmonic mean of a list of numbers skews strongly toward the least elements of the list, it tends (compared to the arithmetic mean) to mitigate the impact of large outliers and aggravate the impact of small ones.
An F1 score punishes extreme values more. Ideally, an F1 Score could be an effective evaluation metric in the following classification scenarios:

- *When FP and FN are equally costly — meaning they miss on true positives or find false positives — both impact the model almost the same way, as in our cancer detection classification example*
- *Adding more data doesn't effectively change the outcome effectively*
- *TN is high (like with flood predictions, cancer predictions, etc.)*

A sample python implementation of the F1 score.

```
import warnings

import pandas

from sklearn import model_selection

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import log_loss

from sklearn.metrics import precision_recall_fscore_support as score, precision_score, recall_score, f
1_score



warnings.filterwarnings('ignore')



url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

dataframe = pandas.read_csv(url)

dat = dataframe.values

X = dat[:,:-1]

y = dat[:,-1]

test_size = 0.33

seed = 7



model = LogisticRegression()

#split data

X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=test_size, random_st
ate=seed)
```

```
model.fit(X_train, y_train)

precision = precision_score(y_test, pred)

print('Precision: %f' % precision)

# recall: tp / (tp + fn)

recall = recall_score(y_test, pred)

print('Recall: %f' % recall)

# f1: tp / (tp + fp + fn)

f1 = f1_score(y_test, pred)

print('F1 score: %f' % f1)
```

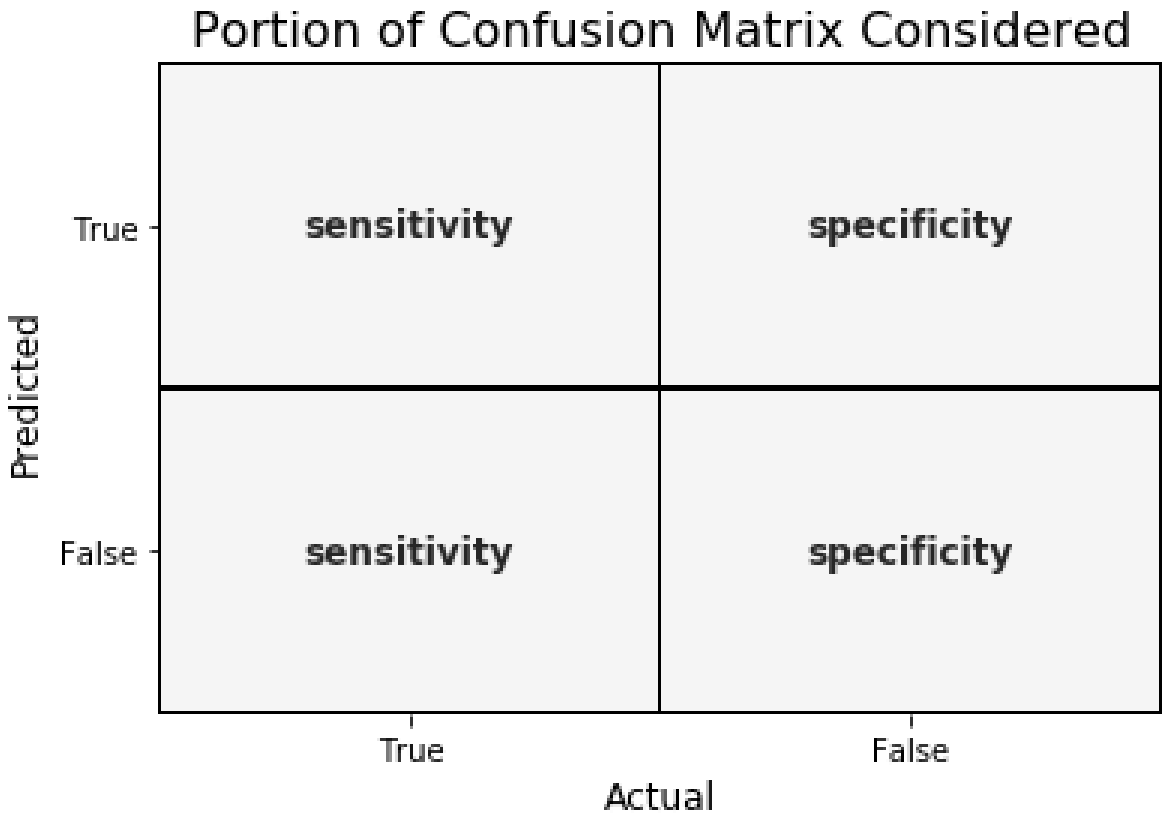
view rawF1-Score.py hosted with ❤ by GitHub

```
Precision: 0.696970
Recall: 0.541176
F1 score: 0.609272
```

5. Specificity

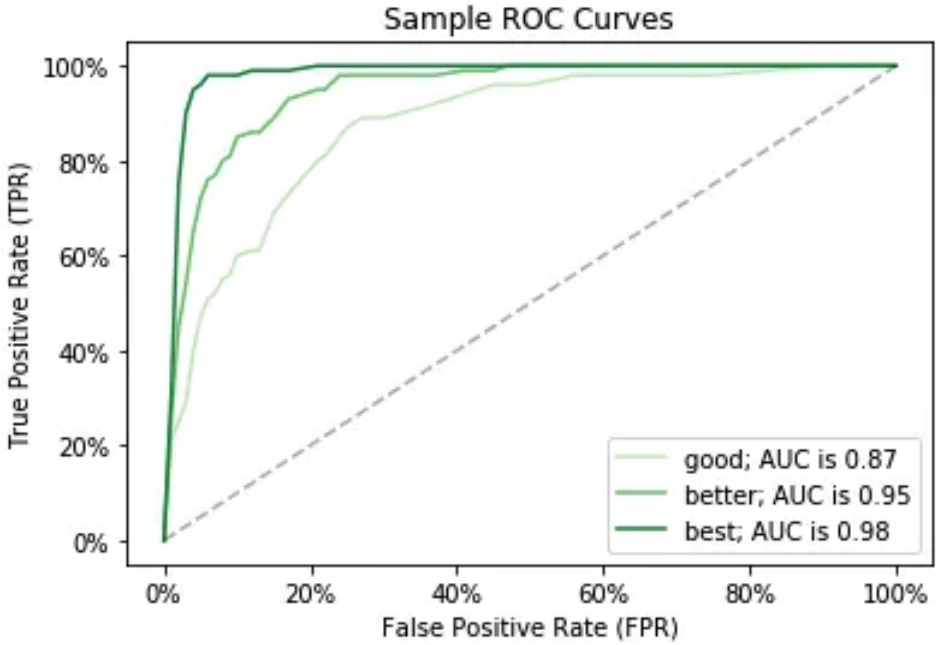
When it's no, how often does it predict no?
True Negative Rate=TN/actual no

It is the **true negative rate** or the proportion of true negatives to everything that should have been classified as negative.
Note that, together, specificity and sensitivity consider the full confusion matrix:



6. Receiver Operating Characteristics (ROC) Curve

Measuring the area under the ROC curve is also a very useful method for evaluating a model. By plotting the true positive rate (sensitivity) versus the false-positive rate (1 — specificity), we get the **Receiver Operating Characteristic** (**ROC**) **curve**. This curve allows us to visualize the trade-off between the true positive rate and the false positive rate.
The following are examples of good ROC curves. The dashed line would be random guessing (no predictive value) and is used as a baseline; anything below that is considered worse than guessing. We want to be toward the top-left corner:

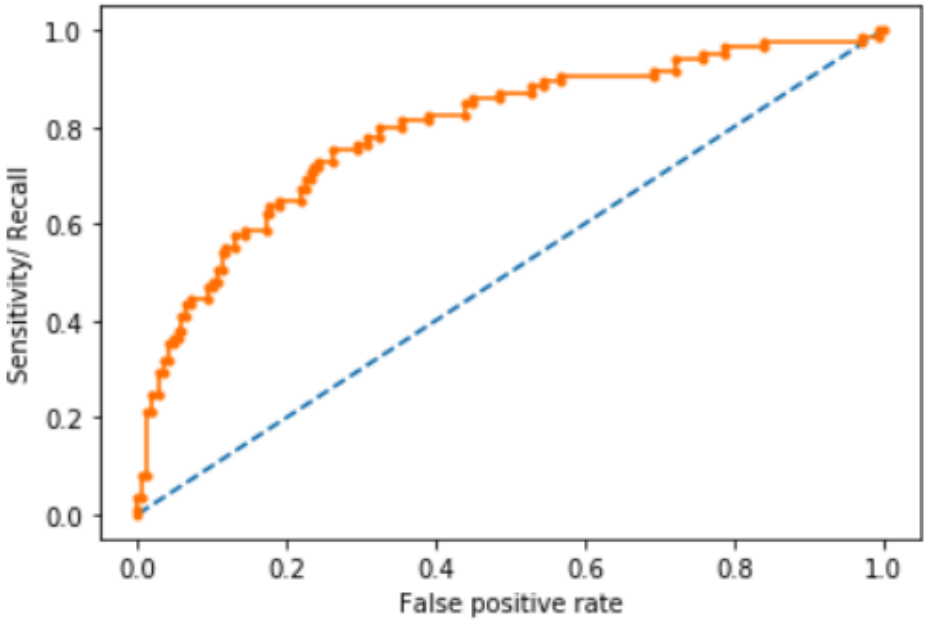


A sample python implementation of the ROC curves.

```
#Classification Area under curve
import warnings
import pandas
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, roc_curve

warnings.filterwarnings('ignore')

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
dataframe = pandas.read_csv(url)
dat = dataframe.values
X = dat[:,:-1]
y = dat[:,-1]
seed = 7
```

```
#split data
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=test_size, random_state=seed)
model.fit(X_train, y_train)


# predict probabilities
probs = model.predict_proba(X_test)
# keep probabilities for the positive outcome only
probs = probs[:, 1]


auc = roc_auc_score(y_test, probs)
print('AUC - Test Set: %.2f%%' % (auc*100))


# calculate roc curve
fpr, tpr, thresholds = roc_curve(y_test, probs)
# plot no skill
plt.plot([0, 1], [0, 1], linestyle='--')
# plot the roc curve for the model
plt.plot(fpr, tpr, marker='.')
plt.xlabel('False positive rate')
plt.ylabel('Sensitivity/ Recall')
# show the plot
plt.show()
```

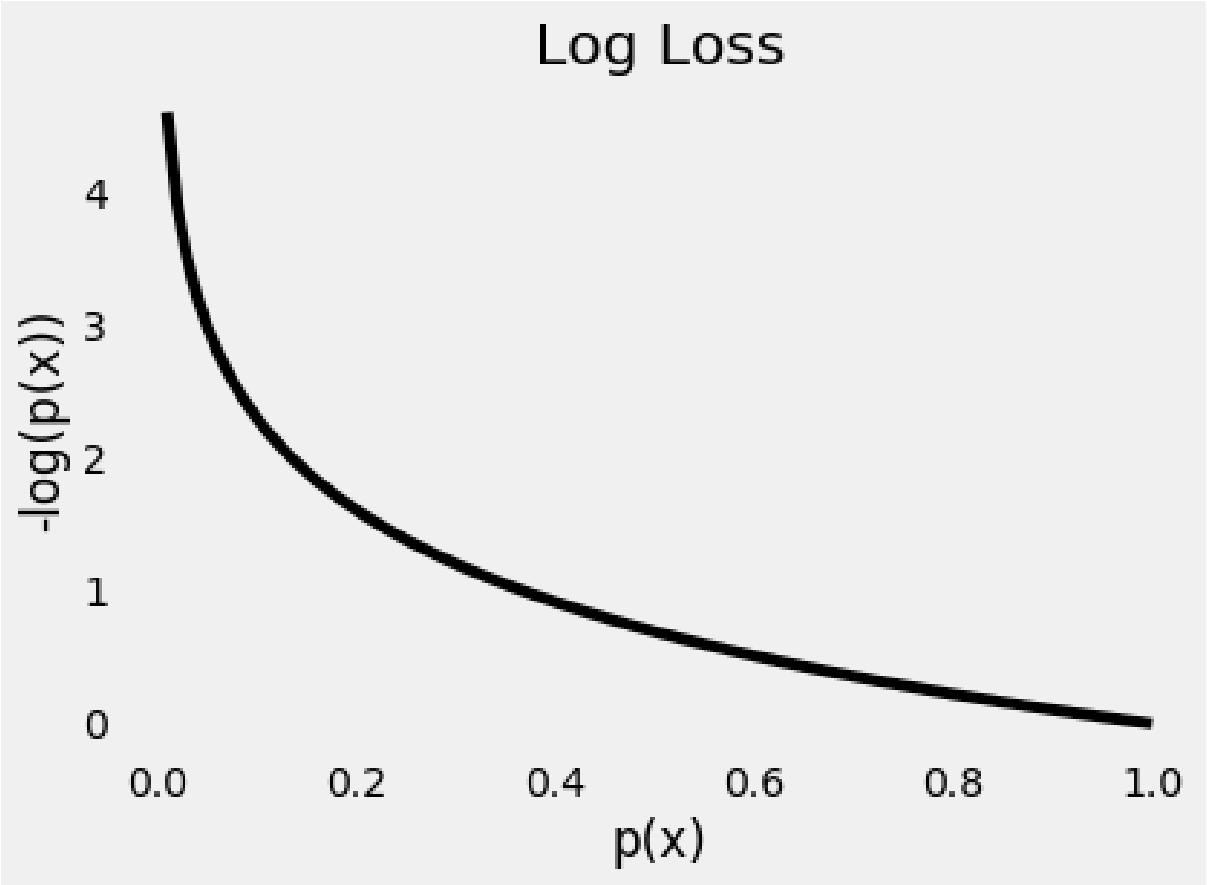
```
AUC - Test Set: 79.65%
```



In the example above, the AUC is relatively close to 1 and greater than 0.5. A perfect classifier will have the ROC curve go along the Y-axis and then along the X-axis.

Log Loss

Log Loss is the most important classification metric based on probabilities.



As the **predicted probability** of the **true class** gets **closer to zero**, the **loss increases exponentially**

It measures the performance of a classification model where the prediction input is a probability value between 0 and 1. Log loss increases as the predicted probability diverge from the actual label. The goal of any machine learning model is to minimize this value. As such, smaller log loss is better, with a perfect model having a log loss of 0.

A sample python implementation of the Log Loss.

```
#Classification LogLoss

import warnings

import pandas

from sklearn import model_selection

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import log_loss



warnings.filterwarnings('ignore')

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
```

```
dataframe = pandas.read_csv(url)

dat = dataframe.values

X = dat[:,:-1]

y = dat[:,-1]

seed = 7

#split data

X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=test_size, random_st
ate=seed)

model.fit(X_train, y_train)

#predict and compute logloss

pred = model.predict(X_test)

accuracy = log_loss(y_test, pred)

print("Logloss: %.2f" % (accuracy))
```

view rawlogg_loss.py hosted with ❤ by GitHub

```
Logloss: 8.02
```

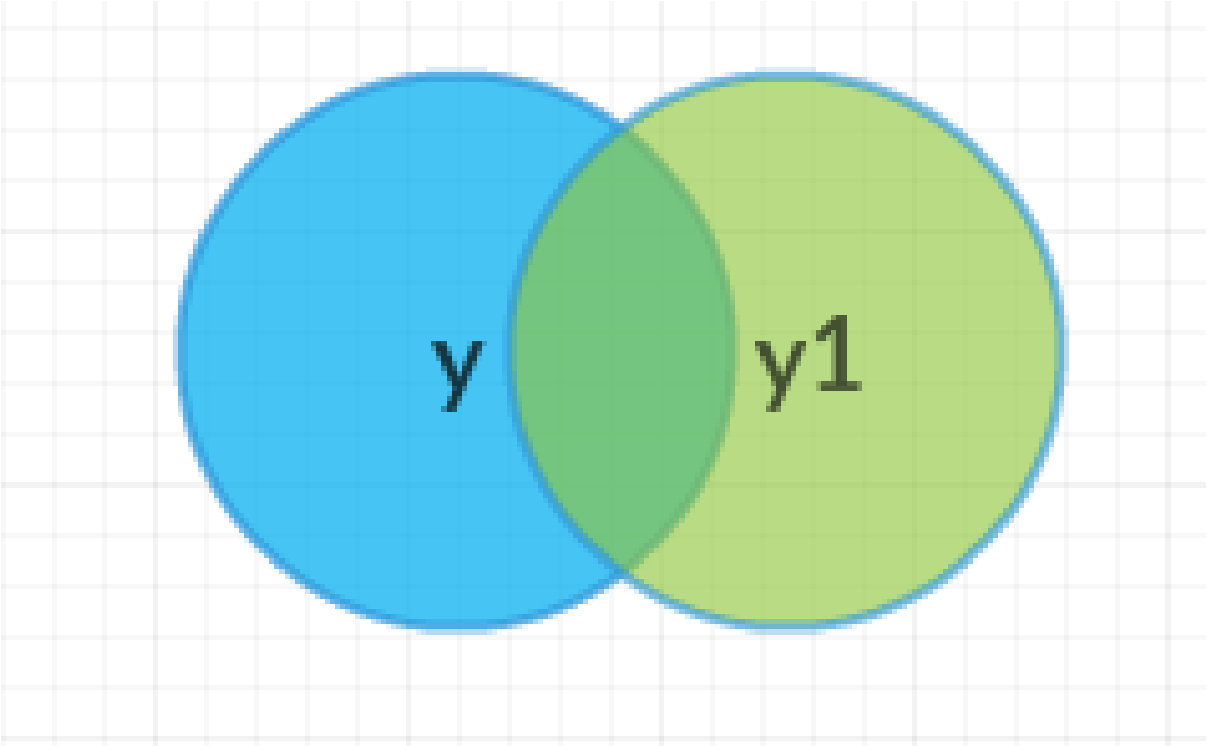
Jaccard Index

Jaccard Index is one of the simplest ways to calculate and find out the accuracy of a classification ML model. Let's understand it with an example. Suppose we have a labeled test set, with labels as –

```
y = [0,0,0,0,0,1,1,1,1,1]
```

And our model has predicted the labels as –

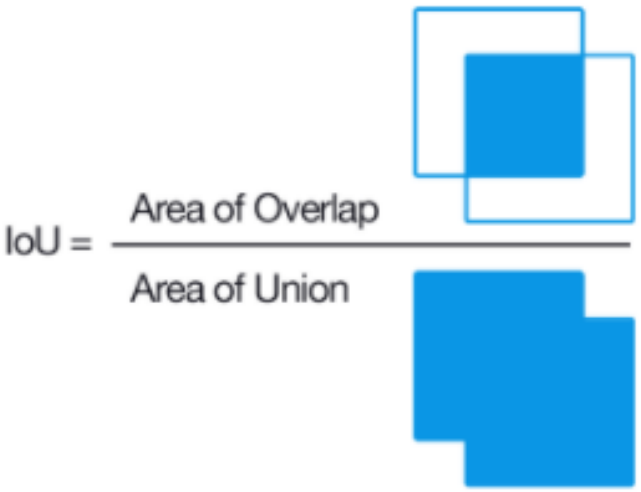
```
y1 = [1,1,0,0,0,1,1,1,1,1]
```



The above Venn diagram shows us the labels of the test set and the labels of the predictions, and their intersection and union.

Jaccard Index or Jaccard similarity coefficient is a statistic used in understanding the similarities between sample sets. The measurement emphasizes the similarity between finite sample sets and is formally defined as the size of the intersection divided by the size of the union of the two labeled sets, with formula as –

$$J(y, y1) = \frac{|y \cap y1|}{|y \cup y1|} = \frac{|y \cap y1|}{|y| + |y1| - |y \cap y1|}$$



Jaccard Index or Intersection over Union(IoU)

So, for our example, we can see that the intersection of the two sets is equal to 8 (since eight values are predicted correctly) and the union is **10 + 10–8 = 12**. So, the Jaccard index gives us the accuracy as –

$$J(y, y1) = \frac{8}{12} = 0.66$$

So, the accuracy of our model, according to **Jaccard Index**, becomes 0.66, or 66%.
Higher the Jaccard index higher the accuracy of the classifier.

A sample python implementation of the Jaccard index.

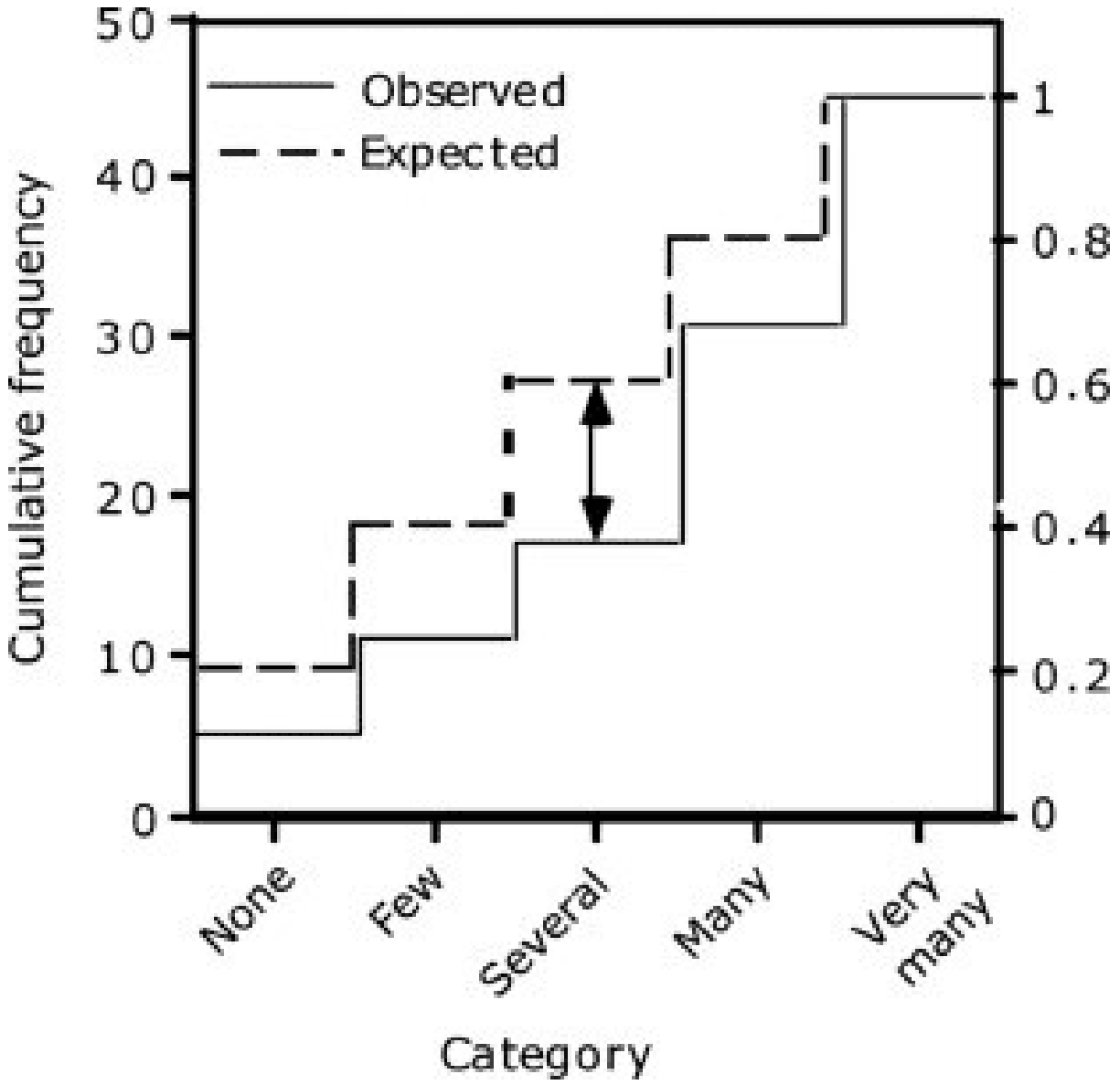
```
import numpy as np


def compute_jaccard_similarity_score(x, y):

intersection_cardinality = len(set(x).intersection(set(y)))

union_cardinality = len(set(x).union(set(y)))

return intersection_cardinality / float(union_cardinality)


score = compute_jaccard_similarity_score(np.array([0, 1, 2, 5, 6]), np.array([0, 2, 3, 5, 7, 9]))

print "Jaccard Similarity Score : %s" %score

pass
```

view rawjaccard_index.py hosted with ❤ by GitHub

```
Jaccard Similarity Score : 0.375
```

Kolomogorov Smirnov chart

K-S or Kolmogorov-Smirnov chart measures the performance of classification models. More accurately, K-S is a measure of the degree of separation between positive and negative distributions.



The cumulative frequency for the observed and hypothesized distributions is plotted against the ordered frequencies. The vertical double arrow indicates the maximal vertical difference.

The K-S is 100 if the scores partition the population into two separate groups in which one group contains all the positives and the other all the negatives. On the other hand, If the model cannot differentiate between positives and negatives, then it is as if the model selects cases randomly from the population. The K-S would be 0.
In most classification models the K-S will fall between 0 and 100, and that the higher the value the better the model is at separating the positive from negative cases.

The K-S may also be used to test whether two underlying one-dimensional probability distributions differ. It is a very efficient way to determine if two samples are significantly different from each other.

A sample python implementation of the Kolmogorov-Smirnov.

```
from scipy.stats import kstest

import random


# N = int(input("Enter number of random numbers: "))

N = 10


actual =[]

print("Enter outcomes: ")


for i in range(N):

# x = float(input("Outcomes of class "+str(i + 1)+":
"))

actual.append(random.random())


print(actual)

x = kstest(actual, "norm")

print(x)
```

view rawKolmogorov-Smirnov.py hosted with ❤ by GitHub

```
Enter outcomes:
[0.44327990912764315, 0.6605825795691285, 0.26203783048580187, 0.306659267902726, 0.2977782138392654, 0.5574513908838
086, 0.9644789110570309, 0.11655821363290042, 0.40892065420391954, 0.9444452718151184]
KstestResult(statistic=0.5463949236854183, pvalue=0.002504270027532512)
```

The Null hypothesis used here assumes that the numbers follow the normal distribution. It returns statistics and p-value. If the **p-value is < alpha**, we reject the Null hypothesis.
Alpha is defined as the probability of rejecting the null hypothesis given the null hypothesis(H0) is true. For most of the practical applications, alpha is chosen as 0.05.

Gain and Lift Chart

Gain or Lift is a measure of the effectiveness of a classification model calculated as the ratio between the results obtained with and without the model. Gain and lift charts are visual aids for evaluating the performance of classification models. However, in contrast to the confusion matrix that evaluates models on the whole population gain or lift chart evaluates model performance in a portion of the population.
The higher the lift (i.e. the further up it is from the baseline), the better the model.

The following gains chart, run on a validation set, shows that with 50% of the data, the model contains 90% of targets, Adding more data adds a negligible increase in the percentage of targets included in the model.

Gain/lift chart

Lift charts are often shown as a **cumulative lift chart**, which is also known as a **gains chart**. Therefore, gains charts are sometimes (perhaps confusingly) called “lift charts”, but they are more accurately *cumulative* lift charts.
It is one of their most common uses is in marketing, to decide if a prospective client is worth calling.

Gini Coefficient

The Gini coefficient or Gini Index is a popular metric for imbalanced class values. The coefficient ranges from 0 to 1 where 0 represents perfect equality and 1 represents perfect inequality. Here, if the value of an index is higher, then the data will be more dispersed.
Gini coefficient can be computed from the area under the ROC curve using the following formula:

Gini Coefficient = (2 * ROC_curve) — 1

What is Feature Engineering

Feature engineering is a machine learning technique that leverages data to create new variables that aren't in the training set. It can produce new features for both supervised and unsupervised learning, with the goal of **simplifying and speeding up data transformations** while also **enhancing model accuracy**. Feature engineering is required when working with machine learning models. Regardless of the data or architecture, a terrible feature will have a direct impact on your model.

Now to understand it in a much easier way, let's take a **simple example**. Below are the prices of properties in x city. It shows the area of the house and total price.

Sq Ft.	Amount
2400	9 Million
3200	15 Million
2500	10 Million
2100	1.5 Million
2500	8.9 Million

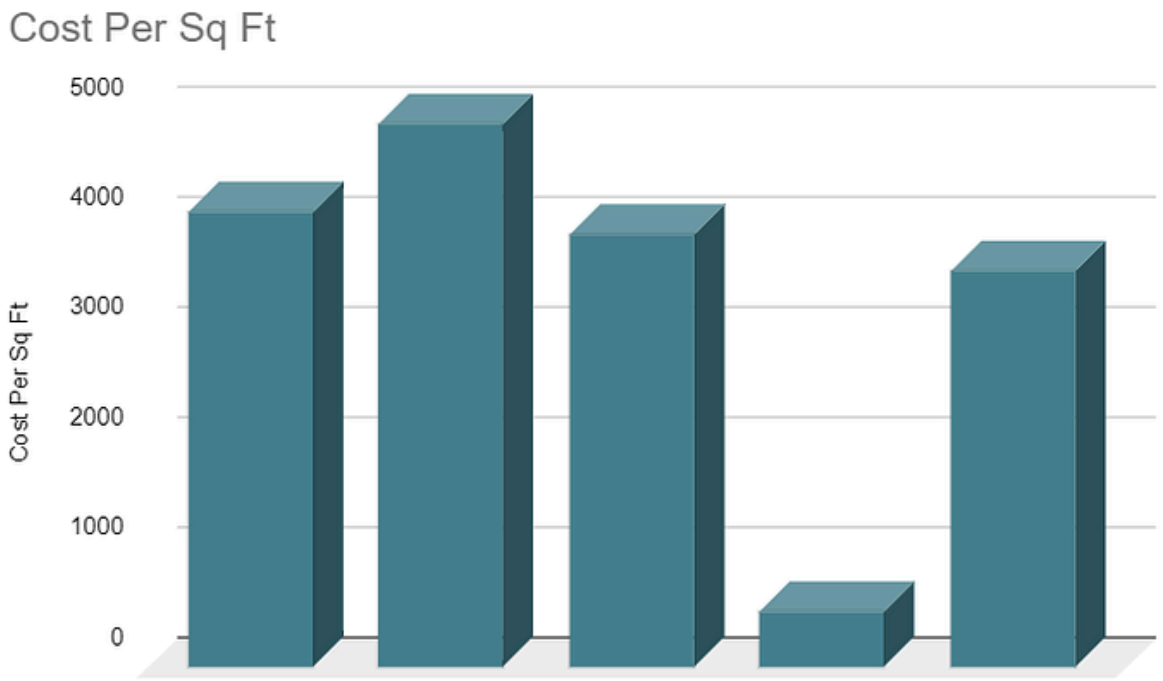
Sample Data

Now this data might have some errors or might be incorrect, not all sources on the internet are correct. To begin, we'll add a new column to display the cost per square foot.

Sq Ft.	Amount	Cost Per Sq Ft
2400	9 Million	4150
3200	15 Million	4944
2500	10 Million	3950
2100	1.5 Million	510
2500	8.9 Million	3600

Sample Data

This new feature will help us understand a lot about our data. So, we have a new column which shows cost per square ft. There are **three main ways** you can find any error. You can use **Domain Knowledge** to contact a property advisor or real estate agent and show him the per square foot rate. If your counsel states that pricing per square foot cannot be less than 3400, you may have a problem. The data can be **visualised**.



When you plot the data, you'll notice that one price is significantly different from the rest. In the **visualisation method**, you can readily notice the problem. The third way is to use **Statistics** to analyze your data and find any problem. Feature engineering consists of various process -

- **Feature Creation**: Creating features involves creating new variables which will be most helpful for our model. This can be adding or removing some features. As we saw above, the cost per sq. ft column was a feature creation.

- **Transformations**: Feature transformation is simply a function that transforms features from one representation to another. The goal here is to plot and visualise data, if something is not adding up with the new features we can reduce the number of features used, speed up training, or increase the accuracy of a certain model.

- **Feature Extraction**: Feature extraction is the process of extracting features from a data set to identify useful information. Without distorting the original relationships or significant information, this compresses the amount of data into manageable quantities for algorithms to process.

- **Exploratory Data Analysis :** Exploratory data analysis (EDA) is a powerful and simple tool that can be used to improve your understanding of your data, by exploring its properties. The technique is often applied when the goal is to create new hypotheses or find patterns in the data. It's often used on large amounts of qualitative or quantitative data that haven't been analyzed before.

- **Benchmark** : A Benchmark Model is the most user-friendly, dependable, transparent, and interpretable model against which you can measure your own. It's a good idea to run test datasets to see if your new machine learning model outperforms a recognised benchmark. These benchmarks are often used as measures for comparing the performance between different machine learning models like neural networks and

support vector machines, linear and non-linear classifiers, or different approaches like bagging and boosting. To learn more about feature engineering steps and process, check the links provided at the end of this article. Now, let's have a look at why we need feature engineering in machine learning.

Importance Of Feature Engineering

Feature Engineering is a very important step in machine learning. Feature engineering refers to the process of designing artificial features into an algorithm. These artificial features are then used by that algorithm in order to improve its performance, or in other words reap better results. Data scientists spend most of their time with data, and it becomes important to make models accurate.

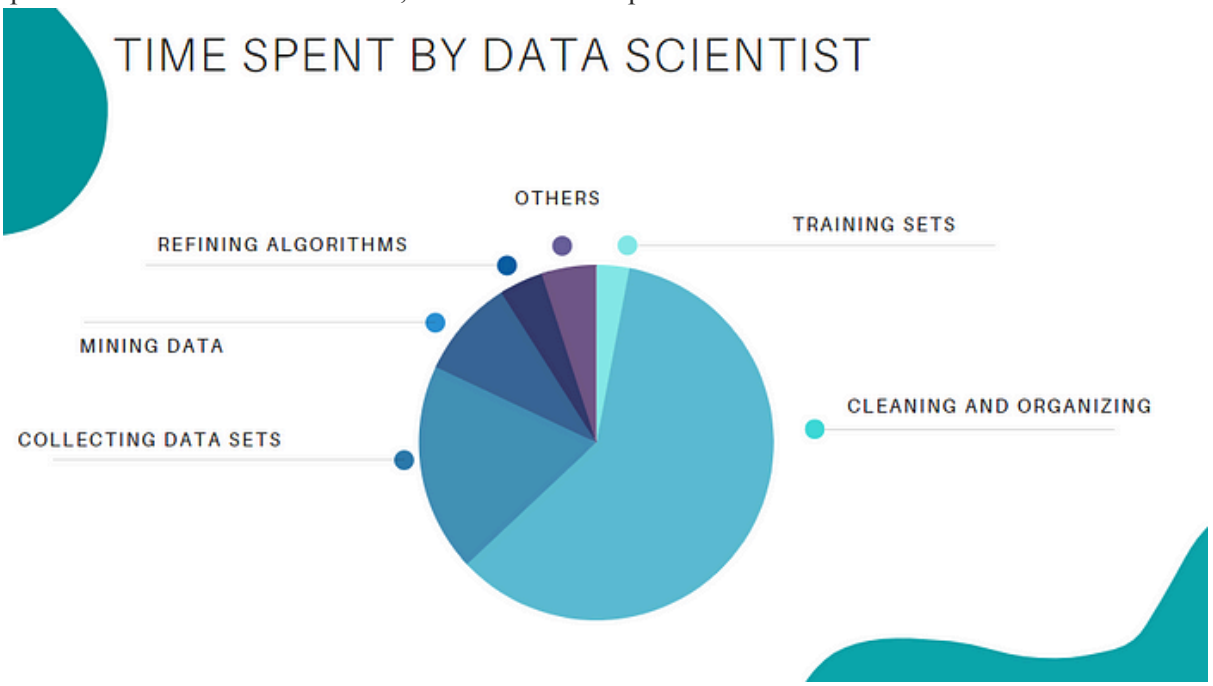


Image By Author

When feature engineering activities are done correctly, the resulting dataset is optimal and contains all of the important factors that affect the business problem. As a result of these datasets, the most accurate predictive models and the most useful insights are produced.

Feature Engineering Techniques for Machine Learning

Lets see a few feature engineering best techniques that you can use. Some of the techniques listed may work better with certain algorithms or datasets, while others may be useful in all situations.

1.Imputation

When it comes to preparing your data for machine learning, missing values are one of the most typical issues. Human errors, data flow interruptions, privacy concerns, and other factors could all contribute to missing values. Missing values have an impact on the performance of machine learning models for

whatever cause. The main goal of imputation is to handle these missing values. There are two types of imputation :

- **Numerical Imputation**: To figure out what numbers should be assigned to people currently in the population, we usually use data from completed surveys or censuses. These data sets can include information about how many people eat different types of food, whether they live in a city or country with a cold climate, and how much they earn every year. That is why numerical imputation is used to fill gaps in surveys or censuses when certain pieces of information are missing.

#Filling all missing values with 0

data = data.fillna(0)

- **Categorical Imputation:** When dealing with categorical columns, replacing missing values with the highest value in the column is a smart solution. However, if you believe the values in the column are evenly distributed and there is no dominating value, imputing a category like "Other" would be a better choice, as your imputation is more likely to converge to a random selection in this scenario.

#Max fill function for categorical columns

data['column_name'].fillna(data['column_name'].value_counts().idxmax(), inplace=True)

2.Handling Outliers

Outlier handling is a technique for removing outliers from a dataset. This method can be used on a variety of scales to produce a more accurate data representation. This has an impact on the model's performance. Depending on the model, the effect could be large or minimal; for example, linear regression is particularly susceptible to outliers. This procedure should be completed prior to model training. The various methods of handling outliers include:

1. **Removal**: Outlier-containing entries are deleted from the distribution. However, if there are outliers across numerous variables, this strategy may result in a big chunk of the datasheet being missed.

2. **Replacing values**: Alternatively, the outliers could be handled as missing values and replaced with suitable imputation.

3. **Capping**: Using an arbitrary value or a value from a variable distribution to replace the maximum and minimum values.

4. **Discretization :** Discretization is the process of converting continuous variables, models, and functions into discrete ones. This is accomplished by constructing a series of continuous intervals (or bins) that span the range of our desired variable/model/function.

3.Log Transform

Log Transform is the most used technique among data scientists. It's mostly used to turn a skewed distribution into a normal or less-skewed distribution. We take the log of the values in a column and utilise those values as the column in this transform. It is used to handle confusing data, and the data becomes more approximative to normal applications.

//Log Example

df[log_price] = np.log(df['Price'])

4.One-hot encoding

A one-hot encoding is a type of encoding in which an element of a finite set is represented by the index in that set, where only one element has its index set to "1" and all other elements are assigned indices within the range [0, n-1]. In contrast to binary encoding schemes, where each bit can represent 2 values (i.e. 0 and 1), this scheme assigns a unique value for each possible case.

5.Scaling

Feature scaling is one of the most pervasive and difficult problems in machine learning, yet it's one of the most important things to get right. In order to train a predictive model, we need data with a known set of features that needs to be scaled up or down as appropriate. This blog post will explain how feature scaling works and why it's important as well as some tips for getting started with feature scaling.

After a scaling operation, the continuous features become similar in terms of range. Although this step isn't required for many algorithms, it's still a good idea to do so. Distance-based algorithms like k-NN and k-Means, on the other hand, require scaled continuous features as model input. There are two common ways for scaling :

Normalization : All values are scaled in a specified range between 0 and 1 via normalisation (or min-max normalisation). This modification has no influence on the feature's distribution, however it does exacerbate the effects of outliers due to lower standard deviations. As a result, it is advised that outliers be dealt with prior to normalisation.

Standardization: Standardization (also known as z-score normalisation) is the process of scaling values while accounting for standard deviation. If the standard deviation of features differs, the range of those features will likewise differ. The effect of outliers in the characteristics is reduced as a result. To arrive at a distribution with a 0 mean and 1 variance, all the data points are subtracted by their mean and the result divided by the distribution's variance.

Learn More about Feature Engineering Techniques

Few Best Feature Engineering Tools

There are many tools which will help you in automating the entire feature engineering process and producing a large pool of features in a short period of time for both classification and regression tasks. So let's have a look at some of the features engineering tools.

FeatureTools

Featuretools is a framework to perform automated feature engineering. It excels at transforming temporal and relational datasets into feature matrices for machine learning. Featuretools integrates with the machine learning pipeline-building tools you already have. In a fraction of the time it would take to do it manually, you can load in pandas dataframes and automatically construct significant features.

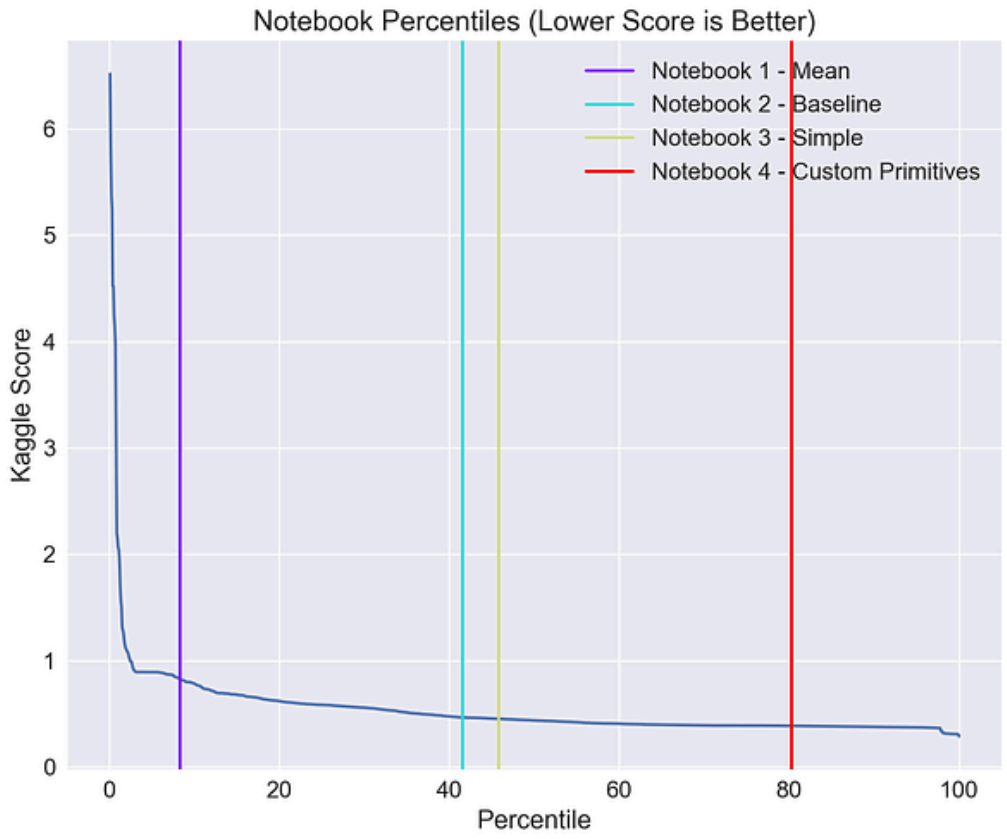


Image Source

FeatureTools Summary

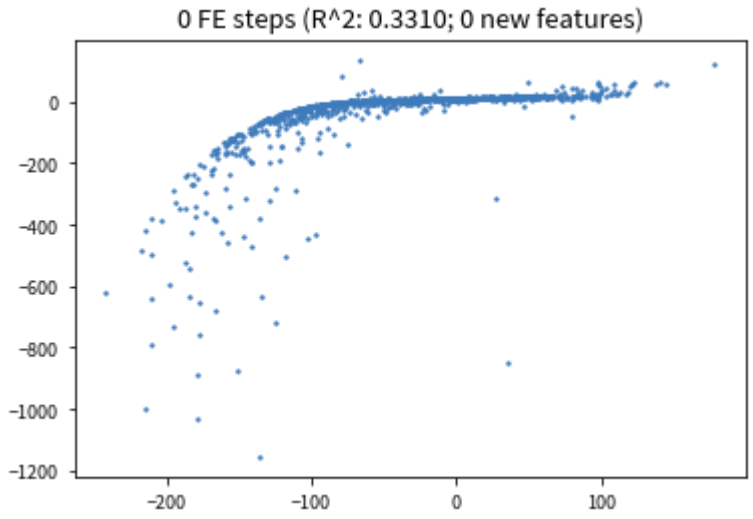
- Easy to get started, good documentation and community support
- It helps you construct meaningful features for machine learning and predictive modelling by combining your raw data with what you know about your data.

- It provides APIs to verify that only legitimate data is utilised for calculations, preventing label leakage in your feature vectors.
- Featuretools includes a low-level function library that may be layered to generate features.
- Its AutoML library(EvalML) helps you build, optimize, and evaluate machine learning pipelines.
- Good at handling relational databases.

Learn More About [FeatureTools].

AutoFeat

AutoFeat helps to perform Linear Prediction Models with Automated Feature Engineering and Selection. AutoFeat allows you to select the units of the input variables in order to avoid the construction of physically nonsensical features.



Source

AutoFeat Summary

- AutoFeat can easily handle categorical features with One hot encoding.
- The AutoFeatRegressor and AutoFeatClassifier models in this package have a similar interface to scikit-learn models
- General purpose automated feature engineering which is Not good at handling relational data.
- It is useful in logistical data

Learn More About AutoFeat.

TsFresh

tsfresh is a python package. It calculates a huge number of time series characteristics, or features, automatically. In addition, the package includes methods for assessing the explanatory power and significance of such traits in regression and classification tasks.

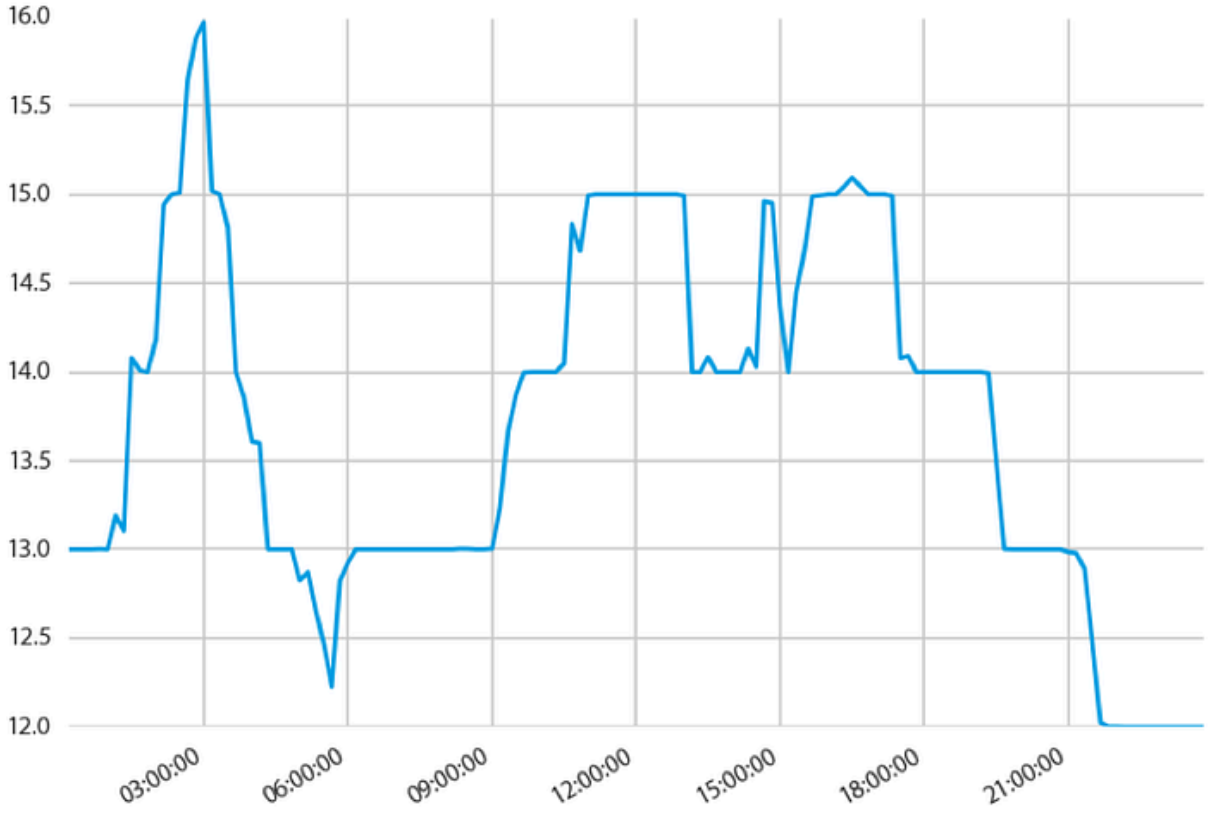


Image Source